

Министерство образования и науки Российской Федерации
федеральное государственное автономное образовательное учреждение высшего образования

**“ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИТМО”**

Факультет прикладной информатики (фПИИ)

Направление подготовки (специальность) – 45.03.04 (Языковые модели и
Искусственный Интеллект)

Алгоритмы и структуры данных

Лабораторная работа № 2

Вариант 21

Выполнил студент: Попов Глеб Ильич Группа № К3260

Преподаватель: Харлунин Александр Александрович

г. Санкт-Петербург 2026 г

Цель:.....	5
Задача №1. Обход двоичного дерева.....	5
Задача №2. Гирлянда.....	8
Задача №3. Простейшее BST.....	11
Задача №4. Простейший неявный ключ.....	16
Задача №5. Простое двоичное дерево поиска.....	20
Задача №6. Опознание двоичного дерева поиска.....	26
Задача №8. Высота дерева возвращается.....	29
Задача №10. Проверка корректности.....	32
Задача №11. Сбалансированное двоичное дерево поиска.....	37
Задача №12. Проверка сбалансированности.....	43
Задача №16. К-й максимум.....	47
Общий вывод по работе.....	53

Цель:

Целью данной лабораторной работы является изучение и практическое применение двоичных деревьев поиска, а также основных операций над ними. В ходе выполнения работы были реализованы алгоритмы обхода дерева, проверки корректности бинарного дерева поиска, вычисления высоты и баланса дерева, а также структуры данных для обработки запросов добавления, удаления, поиска следующего элемента и k-го максимума.

В соответствии с вариантом 21 были решены основные задачи №1, №12 и №16. Дополнительно были выполнены задачи №2, №3, №4, №5, №6, №8, №10 и №11.

Задача №1. Обход двоичного дерева

Текст задачи:

Дано корневое двоичное дерево. Требуется вывести три основных обхода дерева в глубину:

- симметричный обход (in-order),
- прямой обход (pre-order),
- обратный обход (post-order).

Входные данные содержат количество вершин дерева и описание каждой вершины: ключ, индекс левого ребёнка и индекс правого ребёнка. Если ребёнка нет, вместо его номера указано -1.

Нужно вывести три строки:

- ключи вершин в порядке in-order,
- ключи вершин в порядке pre-order,
- ключи вершин в порядке post-order.

Код:

```
import time
import tracemalloc

def solve():
    with open("input.txt", "r", encoding="utf-8") as f:
        data = f.read().split()

    n = int(data[0])

    key = [0] * n
    left = [0] * n
    right = [0] * n

    pos = 1
    for i in range(n):
        key[i] = int(data[pos])
        left[i] = int(data[pos + 1])
        right[i] = int(data[pos + 2])
        pos += 3

    a1 = []
    a2 = []
    a3 = []

    stack = []
    v = 0

    while stack or v != -1:
        while v != -1:
            stack.append(v)
            v = left[v]
        v = stack.pop()
        a1.append(str(key[v]))
```

```

        v = right[v]

    stack = [0]
    while stack:
        v = stack.pop()
        a2.append(str(key[v]))
        if right[v] != -1:
            stack.append(right[v])
        if left[v] != -1:
            stack.append(left[v])

    stack = [(0, 0)]
    while stack:
        v, t = stack.pop()
        if t == 1:
            a3.append(str(key[v]))
        else:
            stack.append((v, 1))
            if right[v] != -1:
                stack.append((right[v], 0))
            if left[v] != -1:
                stack.append((left[v], 0))

    with open("output.txt", "w", encoding="utf-8") as f:
        f.write(" ".join(a1) + "\n")
        f.write(" ".join(a2) + "\n")
        f.write(" ".join(a3) + "\n")

    tracemalloc.start()
    start = time.perf_counter()
    solve()
    finish = time.perf_counter()
    cur, peak = tracemalloc.get_traced_memory()
    tracemalloc.stop()

    with open("stats.txt", "w", encoding="utf-8") as f:
        f.write(f"time = {finish - start:.6f} sec\n")
        f.write(f"memory = {peak / 1024:.2f} KB\n")

```

Текстовое объяснение решения: В данной задаче необходимо вывести три стандартных обхода двоичного дерева. Так как дерево может быть очень глубоким, использование рекурсии может привести к переполнению стека. Поэтому решение было реализовано итеративно, с использованием собственного стека.

Сначала выполняется симметричный обход. Для этого происходит спуск по левым детям, затем обработка вершины и переход к правому ребёнку.

Далее выполняется прямой обход. В стек сначала кладётся корень, затем при извлечении вершины в ответ записывается её ключ. После этого в стек добавляются правый и левый дети, чтобы левый был обработан раньше.

Затем выполняется обратный обход. Для этого в стек кладется пара: номер вершины и признак обработки. Если вершина еще не обработана, она снова помещается в стек с отметкой, что после детей её нужно записать в ответ. Затем в стек добавляются дети.

После выполнения всех трёх обходов результаты записываются в выходной файл.

Таблица тестирования:

время	память	input	output
0.000830 sec	11.52 KB	5 4 1 2 2 3 4 5 -1 -1 1 -1 -1 3 -1 -1	1 2 3 4 5 4 2 1 3 5 1 3 2 5 4
0.000769 sec	10.61 KB	1 10 -1 -1	10 10 10
0.000853 sec	10.99 KB	3 2 1 2 1 -1 -1 3 -1 -1	1 2 3 2 1 3 1 3 2
0.000863 sec	11.29 KB	4 10 1 -1 5 2 -1 2 3 -1 1 -1 -1	1 2 5 10 10 5 2 1 1 2 5 10

Вывод по задаче

В ходе решения задачи были реализованы три основных обхода двоичного дерева: симметричный, прямой и обратный. Все обходы выполнены без рекурсии, что делает программу устойчивой даже для глубоких деревьев. В результате был получен корректный вывод для всех тестовых примеров.

Задача №2. Гирлянда

Текст задачи:

Гирлянда состоит из n лампочек, подвешенных на общем проводе. Высота первой лампочки равна A . Известно, что каждая внутренняя лампочка находится на 1 мм ниже средней высоты двух соседних лампочек.

Требуется найти минимальную возможную высоту последней лампочки B , при которой все лампочки находятся строго выше земли.

Во входных данных заданы:

- число лампочек n ,
- высота первой лампочки A .

Нужно вывести минимальную высоту последней лампочки с точностью не хуже 10^{-6}

Код:

```
import time
import tracemalloc

def check(n, a, h2):
    h1 = a
    mn = min(h1, h2)

    for i in range(3, n + 1):
        h3 = 2 * h2 - h1 + 2
        if h3 < mn:
            mn = h3
        h1 = h2
        h2 = h3

    return mn > 0, h2
```

```

def solve():
    with open("input.txt", "r", encoding="utf-8") as f:
        data = f.read().split()

    n = int(data[0])
    a = float(data[1])

    left = 0.0
    right = a

    for i in range(100):
        mid = (left + right) / 2
        ok, last = check(n, a, mid)

        if ok:
            right = mid
        else:
            left = mid

    ok, ans = check(n, a, right)

    with open("output.txt", "w", encoding="utf-8") as f:
        f.write(f"{ans:.10f}\n")

tracemalloc.start()
start = time.perf_counter()
solve()
finish = time.perf_counter()
cur, peak = tracemalloc.get_traced_memory()
tracemalloc.stop()

with open("stats.txt", "w", encoding="utf-8") as f:
    f.write(f"time = {finish - start:.6f} sec\n")
    f.write(f"memory = {peak / 1024:.2f} KB\n")

```

Текстовое объяснение решения: Жадный Высоты лампочек связаны формулой, поэтому если известна высота второй лампочки, можно последовательно вычислить высоты всех остальных.

Нужно подобрать такую минимальную высоту второй лампочки, чтобы все лампочки были выше нуля. Для этого используется бинарный поиск:

- выбирается середина отрезка,
- по ней вычисляются все высоты,
- если все высоты положительные, можно попробовать уменьшить значение,
- иначе нужно увеличить.

После завершения бинарного поиска получается минимальное подходящее значение, и по нему вычисляется высота последней лампочки.

Таблица тестирования:

время	память	input	output
0.001071 sec	10.41 KB	8 15	9.7500000000
0.001173 sec	10.39 KB	6 15	1.2500000000
0.001434 sec	10.41 KB	7 15	4.5000000000
0.001262 sec	10.39 KB	5 10	0.6666666667

Вывод по задаче

В данной задаче был использован бинарный поиск по ответу. Такой подход позволяет эффективно подобрать минимальную допустимую высоту второй лампочки и затем вычислить высоту последней. Решение укладывается в ограничения по времени и памяти и даёт ответ с требуемой точностью.

Задача №3. Простейшее BST

Текст задачи:

Необходимо реализовать простейшее бинарное дерево поиска и обрабатывать запросы двух типов:

- + x - добавить число x в дерево, если его там ещё нет;

- $> x$ - вывести минимальный элемент дерева, который строго больше x , либо 0, если такого элемента нет.

Во входном файле каждая строка содержит один запрос.

Для каждого запроса вида $> x$ нужно вывести ответ в отдельной строке.

Код:

```
import time
import tracemalloc
import random

def split(v, x):
    if v == -1:
        return -1, -1

    if key[v] <= x:
        a, b = split(right[v], x)
        right[v] = a
        return v, b
    else:
        a, b = split(left[v], x)
        left[v] = b
        return a, v

def merge(a, b):
    if a == -1:
        return b
    if b == -1:
        return a

    if pr[a] > pr[b]:
        right[a] = merge(right[a], b)
        return a
    else:
        left[b] = merge(a, left[b])
        return b

def exists(v, x):
    while v != -1:
        if key[v] == x:
            return True
        if x < key[v]:
            v = left[v]
        else:
```

```

        v = right[v]
    return False

def add(root, x):
    if exists(root, x):
        return root

    v = len(key)
    key.append(x)
    pr.append(random.randint(1, 10**9))
    left.append(-1)
    right.append(-1)

    a, b = split(root, x)
    return merge(merge(a, v), b)

def get_next(v, x):
    ans = 0

    while v != -1:
        if key[v] > x:
            ans = key[v]
            v = left[v]
        else:
            v = right[v]

    return ans

def solve():
    with open("input.txt", "r", encoding="utf-8") as f:
        lines = f.read().splitlines()

    global key, pr, left, right
    key = []
    pr = []
    left = []
    right = []

    root = -1
    out = []

    for s in lines:
        if not s:
            continue

```

```

    t, x = s.split()
    x = int(x)

    if t == "+":
        root = add(root, x)
    else:
        out.append(str(get_next(root, x)))

with open("output.txt", "w", encoding="utf-8") as f:
    f.write("\n".join(out))

tracemalloc.start()
start = time.perf_counter()
solve()
finish = time.perf_counter()
cur, peak = tracemalloc.get_traced_memory()
tracemalloc.stop()

with open("stats.txt", "w", encoding="utf-8") as f:
    f.write(f"time = {finish - start:.6f} sec\n")
    f.write(f"memory = {peak / 1024:.2f} KB\n")

```

Текстовое объяснение решения: В этой задаче нужно поддерживать вставку элементов и поиск минимального элемента, строго большего заданного числа.

Для решения используется декартово дерево. Оно совмещает свойства бинарного дерева поиска и кучи по случайному приоритету. Благодаря этому дерево остаётся достаточно сбалансированным в среднем случае, и операции выполняются быстро.

При добавлении сначала проверяется, есть ли элемент уже в дереве. Если такого элемента нет, выполняется разбиение дерева на две части:

- в левой части находятся все значения, не превосходящие x ,
- в правой части - все значения больше x .

После этого новая вершина с ключом x сливается с этими двумя частями.

Для запроса $> x$ выполняется обычный спуск по дереву:

- если текущий ключ больше x, он может быть ответом, и нужно попробовать найти меньший подходящий ключ слева;
- иначе нужно идти вправо.

Если подходящего элемента нет, выводится 0.

Таблица тестирования:

время	память	input	output
0.000639 sec	11.88 KB	+ 1 + 3 + 3 > 1 > 2 > 3 + 2 > 1	3 3 0 2
0.000802 sec	11.85 KB	+ 10 + 5 + 20 > 4 > 5 > 10 > 20	5 10 20 0
0.000742 sec	11.69 KB	+ 7 > 1 > 6 > 7 > 8	7 7 0 0
0.000773 sec	12.18 KB	+ 8 + 2 + 15 + 11 + 1 > 0 > 1 > 8 > 14	1 2 11 15

Вывод по задаче

В ходе решения задачи была реализована структура данных для добавления элементов и поиска следующего значения. Использование декартового дерева позволило обрабатывать запросы эффективно и укладываться в ограничения по времени и памяти. Программа корректно работает как с повторными вставками, так и с запросами, для которых подходящего элемента в дереве нет.

Задача №4. Простейший неявный ключ

Текст задачи:

Необходимо реализовать бинарное дерево поиска и поддерживать два типа запросов:

- + x - добавить число x в дерево, если его там ещё нет;
- ? k - вывести k-й по возрастанию элемент дерева.

Во входных данных каждая строка содержит один запрос.

Для каждого запроса вида ? k требуется вывести ответ в отдельной строке.

Код:

```
import time
import tracemalloc
import random

def size(v):
    if v == -1:
        return 0
    return cnt[v]

def update(v):
    if v != -1:
        cnt[v] = size(left[v]) + size(right[v]) + 1

def split(v, x):
    if v == -1:
        return -1, -1
```

```

    if key[v] <= x:
        a, b = split(right[v], x)
        right[v] = a
        update(v)
        return v, b
    else:
        a, b = split(left[v], x)
        left[v] = b
        update(v)
        return a, v

def merge(a, b):
    if a == -1:
        return b
    if b == -1:
        return a

    if pr[a] > pr[b]:
        right[a] = merge(right[a], b)
        update(a)
        return a
    else:
        left[b] = merge(a, left[b])
        update(b)
        return b

def exists(v, x):
    while v != -1:
        if key[v] == x:
            return True
        if x < key[v]:
            v = left[v]
        else:
            v = right[v]
    return False

def add(root, x):
    if exists(root, x):
        return root

    v = len(key)
    key.append(x)
    pr.append(random.randint(1, 10**9))
    left.append(-1)

```

```

right.append(-1)
cnt.append(1)

a, b = split(root, x)
return merge(merge(a, v), b)

def get_k(v, k):
    while v != -1:
        s = size(left[v])

        if k == s + 1:
            return key[v]
        if k <= s:
            v = left[v]
        else:
            k -= s + 1
            v = right[v]

def solve():
    with open("input.txt", "r", encoding="utf-8") as f:
        lines = f.read().splitlines()

    global key, pr, left, right, cnt
    key = []
    pr = []
    left = []
    right = []
    cnt = []

    root = -1
    out = []

    for s in lines:
        if not s:
            continue

        t, x = s.split()
        x = int(x)

        if t == "+":
            root = add(root, x)
        else:
            out.append(str(get_k(root, x)))

    with open("output.txt", "w", encoding="utf-8") as f:
        f.write("\n".join(out))

```

```

tracemalloc.start()
start = time.perf_counter()
solve()
finish = time.perf_counter()
cur, peak = tracemalloc.get_traced_memory()
tracemalloc.stop()

with open("stats.txt", "w", encoding="utf-8") as f:
    f.write(f"time = {finish - start:.6f} sec\n")
    f.write(f"memory = {peak / 1024:.2f} KB\n")

```

Текстовое объяснение решения: В этой задаче, кроме вставки элементов, нужно уметь быстро находить k -й по возрастанию элемент.

Для этого в каждой вершине дополнительно хранится размер её поддерева. Это позволяет понять:

- сколько элементов находится слева,
- является ли текущая вершина нужной,
- или нужно идти в правое поддерево.

Если в левом поддереве находится s элементов, то:

- если $k = s + 1$, ответом является текущая вершина;
- если $k \leq s$, нужно идти влево;
- иначе нужно перейти вправо и уменьшить k на $s + 1$.

Для хранения множества используется декартово дерево. При вставке дубликаты игнорируются.

Таблица тестирования:

время	память	input	output
0.000755 sec	12.11 KB	+ 1 + 4 + 3 + 3 ? 1	1 3 4 3

		? 2 ? 3 + 2 ? 3	
0.000592 sec	11.85 KB	+ 10 + 5 + 20 ? 1 ? 2 ? 3	5 10 20
0.000777 sec	12.15 KB	+ 8 + 2 + 15 + 1 + 11 ? 1 ? 3 ? 5	1 8 15
0.000840 sec	12.16 KB	+ 100 + 50 + 150 + 125 + 175 ? 2 ? 4 ? 5	100 150 175

Вывод по задаче

В ходе решения задачи была реализована структура данных, которая поддерживает вставку элементов и поиск k-го по возрастанию значения. За счёт хранения размеров поддеревьев поиск нужного элемента выполняется быстро. Программа корректно обрабатывает повторные вставки и выдаёт правильные ответы на все тесты.

Задача №5. Простое двоичное дерево поиска

Текст задачи:

Требуется реализовать простое двоичное дерево поиска с поддержкой операций:

- insert x — вставить ключ x;
- delete x — удалить ключ x;
- exists x — проверить наличие ключа x;
- next x — найти минимальный элемент, строго больший x;
- prev x — найти максимальный элемент, строго меньший x.

Если вставляемый ключ уже есть в дереве, ничего делать не нужно. Если удаляемого ключа нет, дерево не изменяется.

Для каждой операции exists, next, prev нужно вывести результат.

Код:

```
import time
import tracemalloc

def insert(v, x):
    if v == -1:
        key.append(x)
        left.append(-1)
        right.append(-1)
        return len(key) - 1

    if x < key[v]:
        left[v] = insert(left[v], x)
    elif x > key[v]:
        right[v] = insert(right[v], x)

    return v

def find_min(v):
    while left[v] != -1:
        v = left[v]
```

```

    return v

def delete(v, x):
    if v == -1:
        return -1

    if x < key[v]:
        left[v] = delete(left[v], x)
        return v

    if x > key[v]:
        right[v] = delete(right[v], x)
        return v

    if left[v] == -1:
        return right[v]

    if right[v] == -1:
        return left[v]

    m = find_min(right[v])
    key[v] = key[m]
    right[v] = delete(right[v], key[m])
    return v

def exists(v, x):
    while v != -1:
        if key[v] == x:
            return True
        if x < key[v]:
            v = left[v]
        else:
            v = right[v]
    return False

def get_next(v, x):
    ans = "none"

    while v != -1:
        if key[v] > x:
            ans = str(key[v])
            v = left[v]
        else:
            v = right[v]

```

```

    return ans

def get_prev(v, x):
    ans = "none"

    while v != -1:
        if key[v] < x:
            ans = str(key[v])
            v = right[v]
        else:
            v = left[v]

    return ans

def solve():
    with open("input.txt", "r", encoding="utf-8") as f:
        lines = f.read().splitlines()

    global key, left, right
    key = []
    left = []
    right = []

    root = -1
    out = []

    for s in lines:
        if not s:
            continue

        t, x = s.split()
        x = int(x)

        if t == "insert":
            root = insert(root, x)
        elif t == "delete":
            root = delete(root, x)
        elif t == "exists":
            out.append("true" if exists(root, x) else "false")
        elif t == "next":
            out.append(get_next(root, x))
        else:
            out.append(get_prev(root, x))

    with open("output.txt", "w", encoding="utf-8") as f:
        f.write("\n".join(out))

```

```
tracemalloc.start()
start = time.perf_counter()
solve()
finish = time.perf_counter()
cur, peak = tracemalloc.get_traced_memory()
tracemalloc.stop()

with open("stats.txt", "w", encoding="utf-8") as f:
    f.write(f"time = {finish - start:.6f} sec\n")
    f.write(f"memory = {peak / 1024:.2f} KB\n")
```

Текстовое объяснение решения: В данной задаче реализуется обычное двоичное дерево поиска.

Операция вставки работает по стандартному правилу: если новый ключ меньше текущего, переход выполняется в левое поддерево, иначе — в правое. Если ключ уже существует, дерево не изменяется.

Операция поиска `exists` также выполняется обычным спуском по дереву.

Для операции `next` `x` нужно найти минимальный элемент, который строго больше `x`. Во время спуска по дереву сохраняется текущий подходящий кандидат. Если текущая вершина больше `x`, она может быть ответом, и нужно попробовать найти ещё меньший подходящий элемент слева. Если нет, переход выполняется вправо.

Для операции `prev` `x` всё делается аналогично, только ищется максимальный элемент, строго меньший `x`.

Удаление реализовано стандартно:

- если вершина не найдена, ничего не меняется;
- если у вершины нет одного из поддеревьев, она заменяется вторым;
- если у вершины есть оба поддерева, в неё переносится минимальный ключ из правого поддерева, после чего эта минимальная вершина удаляется.

Таблица тестирования:

время	память	input	output
0.000862 sec	11.86 KB	insert 2 insert 5 insert 3 exists 2 exists 4 next 4 prev 4 delete 5 next 4 prev 4	true false 5 3 none 3
0.000991 sec	11.60 KB	insert 10 insert 5 insert 15 exists 10 exists 7 next 10 prev 10	true false 15 5
0.000756 sec	11.55 KB	insert 8 insert 3 insert 12 delete 3 exists 3 prev 8 next 8	false none 12
0.000641 sec	11.59 KB	insert 1 insert 2 insert 3 delete 2 exists 2 next 1 prev 3	false 3 1

--	--	--	--

Вывод по задаче

В ходе решения задачи было реализовано простое двоичное дерево поиска с операциями вставки, удаления, проверки существования элемента, а также поиска следующего и предыдущего значения. Программа корректно работает на различных тестах и обрабатывает как успешные, так и неуспешные запросы.

Задача №6. Опознание двоичного дерева поиска

Текст задачи:

Дано двоичное дерево с различными целочисленными ключами. Требуется определить, является ли оно корректным двоичным деревом поиска.

Для каждой вершины должны выполняться условия:

- все ключи в левом поддереве строго меньше ключа вершины;
- все ключи в правом поддереве строго больше ключа вершины.

Если дерево корректно, нужно вывести Correct, иначе - Incorrect.

Пустое дерево также считается корректным.

Код:

```
import time
import tracemalloc

def solve():
    with open("input.txt", "r", encoding="utf-8") as f:
        data = f.read().split()

    n = int(data[0])

    if n == 0:
        with open("output.txt", "w", encoding="utf-8") as f:
            f.write("Correct\n")
```

```

        return

    key = [0] * n
    left = [0] * n
    right = [0] * n

    p = 1
    for i in range(n):
        key[i] = int(data[p])
        left[i] = int(data[p + 1])
        right[i] = int(data[p + 2])
        p += 3

    stack = [(0, None, None)]
    ok = True

    while stack:
        v, mn, mx = stack.pop()

        if mn is not None and key[v] <= mn:
            ok = False
            break

        if mx is not None and key[v] >= mx:
            ok = False
            break

        if right[v] != -1:
            stack.append((right[v], key[v], mx))

        if left[v] != -1:
            stack.append((left[v], mn, key[v]))

    with open("output.txt", "w", encoding="utf-8") as f:
        if ok:
            f.write("Correct\n")
        else:
            f.write("Incorrect\n")

    tracemalloc.start()
    start = time.perf_counter()
    solve()
    finish = time.perf_counter()
    cur, peak = tracemalloc.get_traced_memory()
    tracemalloc.stop()

    with open("stats.txt", "w", encoding="utf-8") as f:

```

```
f.write(f"time = {finish - start:.6f} sec\n")
f.write(f"memory = {peak / 1024:.2f} KB\n")
```

Текстовое объяснение решения: В этой задаче недостаточно просто сравнивать вершину с её левым и правым ребёнком. Нужно проверить, чтобы все вершины в левом поддереве были меньше текущего ключа, а все вершины в правом поддереве — больше.

Для этого при обходе дерева для каждой вершины хранится допустимый диапазон значений:

- нижняя граница,
- верхняя граница.

Если ключ вершины не попадает в этот диапазон, значит дерево некорректно.

Сначала в стек кладется корень дерева и диапазон без ограничений. Затем:

- для левого ребёнка верхняя граница становится равной ключу текущей вершины;
- для правого ребёнка нижняя граница становится равной ключу текущей вершины.

Если на каком-то шаге найдено нарушение, дальнейшая проверка уже не нужна.

Такой способ позволяет корректно проверить всё дерево целиком, а не только соседние вершины.

Таблица тестирования:

время	память	input	output
0.000723 sec	10.53 KB	3 2 1 2 1 -1 -1 3 -1 -1	Correct

0.000830 sec	10.56 KB	3 1 1 2 2 -1 -1 3 -1 -1	Incorrect
0.000583 sec	10.21 KB	0	Correct
0.000652 sec	10.66 KB	4 4 1 -1 2 2 3 1 -1 -1 5 -1 -1	Incorrect

Вывод по задаче

В ходе решения задачи была реализована проверка корректности двоичного дерева поиска с помощью обхода дерева и хранения допустимых границ значений для каждой вершины. Такой подход позволяет правильно учитывать все ограничения на поддеревья. Программа корректно определяет как правильные, так и неправильные деревья, а также обрабатывает случай пустого дерева.

Задача №8. Высота дерева возвращается

Текст задачи:

Дано двоичное дерево поиска. Высотой дерева называется максимальное число вершин в цепочке, начинающейся в корне дерева и заканчивающейся в одном из его листьев.

Необходимо найти высоту данного дерева.

Если дерево пустое, его высота считается равной 0. Если дерево состоит из одной вершины, его высота равна 1.

Во входных данных:

- в первой строке находится число N - количество вершин,
- далее идут описания вершин,
- для каждой вершины указаны её ключ, номер левого ребёнка и номер правого ребёнка,
- если ребёнка нет, вместо его номера указано 0

Код:

```
import time
import tracemalloc

def solve():
    with open("input.txt", "r", encoding="utf-8") as f:
        data = f.read().split()

    n = int(data[0])

    if n == 0:
        with open("output.txt", "w", encoding="utf-8") as f:
            f.write("0\n")
        return

    left = [0] * (n + 1)
    right = [0] * (n + 1)

    p = 1
    for i in range(1, n + 1):
        p += 1
        left[i] = int(data[p])
        right[i] = int(data[p + 1])
        p += 2

    stack = [(1, 1)]
    ans = 0

    while stack:
        v, h = stack.pop()

        if h > ans:
            ans = h

        if left[v] != 0:
            stack.append((left[v], h + 1))

        if right[v] != 0:
```

```

        stack.append((right[v], h + 1))

    with open("output.txt", "w", encoding="utf-8") as f:
        f.write(str(ans) + "\n")

tracemalloc.start()
start = time.perf_counter()
solve()
finish = time.perf_counter()
cur, peak = tracemalloc.get_traced_memory()
tracemalloc.stop()

with open("stats.txt", "w", encoding="utf-8") as f:
    f.write(f"time = {finish - start:.6f} sec\n")
    f.write(f"memory = {peak / 1024:.2f} KB\n")

```

Текстовое объяснение решения: В этой задаче дерево уже задано во входных данных, поэтому ничего вставлять не нужно. Необходимо только определить его высоту.

Для этого выполняется обход дерева, начиная с корня. Вместе с каждой вершиной в стек кладётся текущая глубина. При извлечении вершины из стека значение глубины сравнивается с текущим максимумом. Если оно больше, максимум обновляется.

После этого в стек добавляются левый и правый ребёнок текущей вершины, если они существуют. Для детей глубина увеличивается на 1.

Так как каждую вершину мы посещаем один раз, решение работает за линейное время.

.

Таблица тестирования:

время	память	input	output
0.001063 sec	10.50 KB	6 -2 0 2 8 4 3	4

		9 0 0 3 6 5 6 0 0 0 0 0	
0.000686 sec	10.21 KB	0	0
0.000760 sec	10.31 KB	1 10 0 0	1
0.000878 sec	10.47 KB	4 10 2 0 5 3 0 2 4 0 1 0 0	4

Вывод по задаче

В ходе решения задачи была найдена высота двоичного дерева поиска с помощью итеративного обхода. Программа корректно обрабатывает как пустое дерево, так и дерево из одной вершины, а также более сложные случаи. Решение работает за линейное время и укладывается в заданные ограничения.

Задача №10. Проверка корректности

Текст задачи:

Дано двоичное дерево. Требуется проверить, выполняется ли для него свойство двоичного дерева поиска:

- все ключи в левом поддереве вершины меньше её ключа;

- все ключи в правом поддереве больше её ключа.

Если дерево является двоичным деревом поиска, нужно вывести Yes, иначе - No.

Во входных данных:

- в первой строке находится число N - количество вершин,
- далее идут описания вершин,
- для каждой вершины указаны её ключ, номер левого ребёнка и номер правого ребёнка,
- если ребёнка нет, вместо его номера указано 0.

Пустое дерево также считается корректным.

Код:

```
import time

import tracemalloc

def solve():

    with open("input.txt", "r", encoding="utf-8") as f:

        data = f.read().split()

    n = int(data[0])

    if n == 0:

        with open("output.txt", "w", encoding="utf-8") as f:

            f.write("Yes\n")

        return

    key = [0] * (n + 1)

    left = [0] * (n + 1)
```

```
right = [0] * (n + 1)

p = 1

for i in range(1, n + 1):
    key[i] = int(data[p])
    left[i] = int(data[p + 1])
    right[i] = int(data[p + 2])
    p += 3

stack = [(1, None, None)]
ok = True

while stack:
    v, mn, mx = stack.pop()

    if mn is not None and key[v] <= mn:
        ok = False
        break

    if mx is not None and key[v] >= mx:
        ok = False
        break

    if right[v] != 0:
        stack.append((right[v], key[v], mx))

    if left[v] != 0:
        stack.append((left[v], mn, key[v]))
```

```

with open("output.txt", "w", encoding="utf-8") as f:

    if ok:

        f.write("Yes\n")

    else:

        f.write("No\n")

tracemalloc.start()

start = time.perf_counter()

solve()

finish = time.perf_counter()

cur, peak = tracemalloc.get_traced_memory()

tracemalloc.stop()

with open("stats.txt", "w", encoding="utf-8") as f:

    f.write(f"time = {finish - start:.6f} sec\n")

    f.write(f"memory = {peak / 1024:.2f} KB\n")

```

Текстовое объяснение решения: В этой задаче недостаточно проверить только непосредственных детей каждой вершины. Нужно убедиться, что все элементы левого поддерева меньше ключа вершины, а все элементы правого поддерева больше.

Для этого при обходе дерева для каждой вершины хранится допустимый диапазон значений:

- нижняя граница,
- верхняя граница.

Корень дерева изначально не имеет ограничений.

Если мы переходим в левое поддереву, верхняя граница становится равной ключу текущей вершины.

Если переходим в правое поддереву, нижняя граница становится равной ключу текущей вершины.

Если хотя бы одна вершина выходит за допустимые границы, значит дерево не является корректным двоичным деревом поиска.

Такой подход позволяет правильно проверять всё дерево целиком, а не только связи между соседними вершинами.

Таблица тестирования:

время	память	input	output
0.000904 sec	10.56 KB	6 -2 0 2 8 4 3 9 0 0 3 6 5 6 0 0 0 0 0	Yes
0.000765 sec	10.21 KB	0	Yes
0.000882 sec	10.41 KB	3 5 2 3 6 0 0 4 0 0	No
0.000790 sec	10.38 KB	3 2 2 3 1 0 0 3 0 0	Yes

--	--	--	--

Вывод по задаче

В ходе решения задачи была реализована проверка свойства двоичного дерева поиска с помощью обхода дерева и хранения допустимых границ для каждой вершины. Такой подход позволяет корректно учитывать не только соседние вершины, но и все поддеревья. Программа правильно определяет как корректные, так и некорректные деревья и укладывается в заданные ограничения.

Задача №11. Сбалансированное двоичное дерево поиска

Текст задачи:

Требуется реализовать сбалансированное двоичное дерево поиска с поддержкой операций:

- insert x - вставить ключ x;
- delete x - удалить ключ x;
- exists x - проверить наличие ключа x;
- next x - найти минимальный элемент, строго больший x;
- prev x - найти максимальный элемент, строго меньший x.

Если вставляемый ключ уже есть в дереве, ничего делать не нужно. Если удаляемого ключа нет, дерево не изменяется.

Для каждой операции exists, next, prev нужно вывести результат.

Код:

```
import time
import tracemalloc

def h(v):
```

```

    if v == -1:
        return 0
    return height[v]

def upd(v):
    height[v] = max(h(left[v]), h(right[v])) + 1

def bal(v):
    return h(right[v]) - h(left[v])

def new_node(x):
    key.append(x)
    left.append(-1)
    right.append(-1)
    height.append(1)
    return len(key) - 1

def rotate_left(v):
    u = right[v]
    right[v] = left[u]
    left[u] = v
    upd(v)
    upd(u)
    return u

def rotate_right(v):
    u = left[v]
    left[v] = right[u]
    right[u] = v
    upd(v)
    upd(u)
    return u

def fix(v):
    upd(v)

    if bal(v) == 2:
        if bal(right[v]) < 0:
            right[v] = rotate_right(right[v])
        return rotate_left(v)

    if bal(v) == -2:

```

```

        if bal(left[v]) > 0:
            left[v] = rotate_left(left[v])
        return rotate_right(v)

    return v

def insert(v, x):
    if v == -1:
        return new_node(x)

    if x < key[v]:
        left[v] = insert(left[v], x)
    elif x > key[v]:
        right[v] = insert(right[v], x)

    return fix(v)

def find_min(v):
    while left[v] != -1:
        v = left[v]
    return v

def remove_min(v):
    if left[v] == -1:
        return right[v]

    left[v] = remove_min(left[v])
    return fix(v)

def delete(v, x):
    if v == -1:
        return -1

    if x < key[v]:
        left[v] = delete(left[v], x)
        return fix(v)

    if x > key[v]:
        right[v] = delete(right[v], x)
        return fix(v)

    a = left[v]
    b = right[v]

```

```

    if b == -1:
        return a

    m = find_min(b)
    right[m] = remove_min(b)
    left[m] = a
    return fix(m)

def exists(v, x):
    while v != -1:
        if key[v] == x:
            return True
        if x < key[v]:
            v = left[v]
        else:
            v = right[v]
    return False

def get_next(v, x):
    ans = "none"

    while v != -1:
        if key[v] > x:
            ans = str(key[v])
            v = left[v]
        else:
            v = right[v]

    return ans

def get_prev(v, x):
    ans = "none"

    while v != -1:
        if key[v] < x:
            ans = str(key[v])
            v = right[v]
        else:
            v = left[v]

    return ans

def solve():
    with open("input.txt", "r", encoding="utf-8") as f:

```

```

        lines = f.read().splitlines()

    global key, left, right, height
    key = []
    left = []
    right = []
    height = []

    root = -1
    out = []

    for s in lines:
        if not s:
            continue

        t, x = s.split()
        x = int(x)

        if t == "insert":
            root = insert(root, x)
        elif t == "delete":
            root = delete(root, x)
        elif t == "exists":
            out.append("true" if exists(root, x) else "false")
        elif t == "next":
            out.append(get_next(root, x))
        else:
            out.append(get_prev(root, x))

    with open("output.txt", "w", encoding="utf-8") as f:
        f.write("\n".join(out))

tracemalloc.start()
start = time.perf_counter()
solve()
finish = time.perf_counter()
cur, peak = tracemalloc.get_traced_memory()
tracemalloc.stop()

with open("stats.txt", "w", encoding="utf-8") as f:
    f.write(f"time = {finish - start:.6f} sec\n")
    f.write(f"memory = {peak / 1024:.2f} KB\n")

```

Текстовое объяснение решения: В этой задаче требуется не обычное дерево поиска, а сбалансированное. Для этого было реализовано AVL-дерево.

В каждой вершине хранится её высота. После каждой вставки или удаления высота пересчитывается, а затем вычисляется баланс вершины:

- если баланс равен 2, дерево перевешивает вправо;
- если баланс равен -2, дерево перевешивает влево.

В таких случаях выполняются повороты:

- малый левый,
- малый правый,
- либо двойные повороты, если поддереву наклонено в противоположную сторону.

Операции exists, next и prev выполняются обычным спуском по дереву поиска.

Операция удаления устроена стандартно:

- если вершина имеет одно поддерево или не имеет детей, она заменяется этим поддеревом;
- если у вершины два поддерева, в неё переносится минимальный элемент из правого поддерева, после чего эта вершина удаляется.

За счёт балансировки высота дерева остаётся логарифмической, поэтому все операции работают быстро.

Таблица тестирования:

время	память	input	output
0.001152 sec	11.15 KB	insert 2 insert 5 insert 3 exists 2 exists 4	true false 5 3 none

		next 4 prev 4 delete 5 next 4 prev 4	3
0.000749 sec	10.83 KB	insert 10 insert 20 insert 30 exists 20 prev 20 next 20	true 10 30
0.000723 sec	10.92 KB	insert 8 insert 4 insert 12 insert 10 delete 8 exists 8 next 9 prev 11	False 10 10
0.000947 sec	10.92 KB	insert 1 insert 2 insert 3 insert 4 delete 2 exists 2 prev 3 next 3	False 1 4

Вывод по задаче

В ходе решения задачи было реализовано сбалансированное двоичное дерево поиска - AVL-дерево. Это позволило эффективно выполнять вставку, удаление, поиск, а также операции нахождения следующего и предыдущего элемента. Программа корректно работает на различных тестах и укладывается в заданные ограничения по времени и памяти.

Задача №12. Проверка сбалансированности

Текст задачи:

Дано двоичное дерево поиска. Для каждой его вершины требуется определить её баланс.

Баланс вершины определяется как разность:

высота правого поддерева - высота левого поддерева

Во входных данных:

- в первой строке находится число N - количество вершин,
- далее идут описания вершин,
- для каждой вершины указаны её ключ, номер левого ребёнка и номер правого ребёнка,
- если ребёнка нет, вместо его номера указано 0.

Необходимо для каждой вершины вывести её баланс в отдельной строке. Формат и определение баланса в задаче именно такие.

Код:

```
import time
import tracemalloc

def solve():
    with open("input.txt", "r", encoding="utf-8") as f:
        data = f.read().split()

    n = int(data[0])

    if n == 0:
        with open("output.txt", "w", encoding="utf-8") as f:
            f.write("")
```

```

        return

    left = [0] * (n + 1)
    right = [0] * (n + 1)

    p = 1
    for i in range(1, n + 1):
        p += 1
        left[i] = int(data[p])
        right[i] = int(data[p + 1])
        p += 2

    order = []
    stack = [1]

    while stack:
        v = stack.pop()
        order.append(v)

        if left[v] != 0:
            stack.append(left[v])
        if right[v] != 0:
            stack.append(right[v])

    h = [0] * (n + 1)

    for v in order[::-1]:
        h[v] = max(h[left[v]], h[right[v]]) + 1

    out = []
    for v in range(1, n + 1):
        out.append(str(h[right[v]] - h[left[v]]))

    with open("output.txt", "w", encoding="utf-8") as f:
        f.write("\n".join(out))

tracemalloc.start()
start = time.perf_counter()
solve()
finish = time.perf_counter()
cur, peak = tracemalloc.get_traced_memory()
tracemalloc.stop()

with open("stats.txt", "w", encoding="utf-8") as f:
    f.write(f"time = {finish - start:.6f} sec\n")
    f.write(f"memory = {peak / 1024:.2f} KB\n")

```

Текстовое объяснение решения: В этой задаче нужно для каждой вершины найти баланс. Для этого сначала необходимо узнать высоты всех поддеревьев.

Так как дерево уже задано, выполняется обход от корня и собирается порядок вершин. Затем вершины обрабатываются в обратном порядке, то есть снизу вверх. В этот момент для каждой вершины уже известны высоты её левого и правого поддеревьев, поэтому высоту самой вершины можно вычислить по формуле:

высота вершины = $\max(\text{высота левого поддерева}, \text{высота правого поддерева}) + 1$

После того как высоты всех вершин найдены, для каждой вершины вычисляется баланс:

баланс = высота правого поддерева - высота левого поддерева

Полученные значения последовательно записываются в выходной файл.

Таблица тестирования:

время	память	input	output
0.000881 sec	10.93 KB	6 -2 0 2 8 4 3 9 0 0 3 6 5 6 0 0 0 0 0	3 -1 0 0 0 0
0.000802 sec	10.43 KB	1 10 0 0	0

0.000705 sec	10.56 KB	3 2 2 3 1 0 0 3 0 0	0 0 0
0.000769 sec	10.74 KB	4 10 2 0 5 3 0 2 4 0 1 0 0	-3 -2 -1 0

Вывод по задаче

В ходе решения задачи были вычислены высоты всех поддеревьев, после чего для каждой вершины был найден её баланс. Решение работает за линейное время, так как каждая вершина обрабатывается ограниченное число раз. Программа корректно определяет баланс как для сбалансированных деревьев, так и для вырожденных случаев.

Задача №16. К-й максимум

Текст задачи: Необходимо реализовать структуру данных, которая поддерживает три типа операций:

- 1 x - добавить элемент с ключом x;
- -1 x - удалить элемент с ключом x;
- 0 k - вывести k-й максимум среди всех элементов структуры.

Гарантируется, что:

- одинаковые ключи одновременно хранить не требуется;
- удаляются только существующие элементы;
- при запросе k-го максимума такой элемент всегда существует

Код:

```
import time
import tracemalloc

def h(v):
    if v == -1:
        return 0
    return height[v]

def sz(v):
    if v == -1:
        return 0
    return cnt[v]

def upd(v):
    height[v] = max(h(left[v]), h(right[v])) + 1
    cnt[v] = sz(left[v]) + sz(right[v]) + 1

def bal(v):
    return h(right[v]) - h(left[v])

def new_node(x):
    key.append(x)
    left.append(-1)
    right.append(-1)
    height.append(1)
    cnt.append(1)
    return len(key) - 1

def rotate_left(v):
    u = right[v]
    right[v] = left[u]
    left[u] = v
    upd(v)
    upd(u)
    return u
```

```

def rotate_right(v):
    u = left[v]
    left[v] = right[u]
    right[u] = v
    upd(v)
    upd(u)
    return u

def fix(v):
    upd(v)

    if bal(v) == 2:
        if bal(right[v]) < 0:
            right[v] = rotate_right(right[v])
        return rotate_left(v)

    if bal(v) == -2:
        if bal(left[v]) > 0:
            left[v] = rotate_left(left[v])
        return rotate_right(v)

    return v

def insert(v, x):
    if v == -1:
        return new_node(x)

    if x < key[v]:
        left[v] = insert(left[v], x)
    elif x > key[v]:
        right[v] = insert(right[v], x)

    return fix(v)

def find_min(v):
    while left[v] != -1:
        v = left[v]
    return v

def remove_min(v):
    if left[v] == -1:
        return right[v]

```

```

    left[v] = remove_min(left[v])
    return fix(v)

def delete(v, x):
    if v == -1:
        return -1

    if x < key[v]:
        left[v] = delete(left[v], x)
        return fix(v)

    if x > key[v]:
        right[v] = delete(right[v], x)
        return fix(v)

    a = left[v]
    b = right[v]

    if b == -1:
        return a

    m = find_min(b)
    right[m] = remove_min(b)
    left[m] = a
    return fix(m)

def get_k_max(v, k):
    while v != -1:
        s = sz(right[v])

        if k == s + 1:
            return key[v]
        if k <= s:
            v = right[v]
        else:
            k -= s + 1
            v = left[v]

def solve():
    with open("input.txt", "r", encoding="utf-8") as f:
        data = f.read().split()

    n = int(data[0])

    global key, left, right, height, cnt

```

```

key = []
left = []
right = []
height = []
cnt = []

root = -1
out = []

p = 1
for i in range(n):
    t = int(data[p])
    x = int(data[p + 1])
    p += 2

    if t == 1:
        root = insert(root, x)
    elif t == -1:
        root = delete(root, x)
    else:
        out.append(str(get_k_max(root, x)))

with open("output.txt", "w", encoding="utf-8") as f:
    f.write("\n".join(out))

tracemalloc.start()
start = time.perf_counter()
solve()
finish = time.perf_counter()
cur, peak = tracemalloc.get_traced_memory()
tracemalloc.stop()

with open("stats.txt", "w", encoding="utf-8") as f:
    f.write(f"time = {finish - start:.6f} sec\n")
    f.write(f"memory = {peak / 1024:.2f} KB\n")

```

Текстовое объяснение решения: В этой задаче нужно не только добавлять и удалять элементы, но и быстро находить k-й максимум. Для этого обычного дерева поиска недостаточно, потому что в худшем случае оно может превратиться в цепочку. Поэтому было реализовано AVL-дерево.

Кроме высоты вершины, в каждой вершине хранится ещё и размер её поддерева. Это позволяет быстро понять, сколько элементов находится справа от текущей вершины.

Если в правом поддереве вершины находится s элементов, тогда:

- текущая вершина является $(s + 1)$ -й по убыванию;
- если нужно найти меньший номер, надо идти вправо;
- если больший - идти влево, уменьшив k на $s + 1$.

После каждой вставки и удаления пересчитываются:

- высота вершины,
- размер поддерева,
- баланс вершины.

Если баланс выходит за пределы допустимого диапазона, выполняются повороты, как в обычном AVL-дереве. За счёт этого дерево остаётся сбалансированным, и все операции работают быстро.

.

Таблица тестирования:

время	память	input	output
0.000855 sec	11.01 KB	11 1 5 1 3 1 7 0 1 0 2 0 3 -1 5 1 10 0 1 0 2 0 3	7 5 3 10 7 3

0.000652 sec	10.88 KB	7 1 10 1 20 1 30 0 1 0 2 0 3 -1 20	30 20 10
0.000733 sec	10.79 KB	8 1 8 1 2 1 15 1 10 0 1 0 2 0 3 0 4	15 10 8 2
0.000626 sec	11.12 KB	9 1 50 1 30 1 70 1 60 -1 70 0 1 0 2 0 3 1 80	60 50 30

Вывод по задаче

В ходе решения задачи была реализована структура данных для добавления, удаления и поиска k -го максимума. За счёт использования AVL-дерева и хранения размеров поддеревьев все операции выполняются эффективно. Программа корректно работает на различных тестах и укладывается в ограничения по времени и памяти.

Общий вывод по работе

В ходе выполнения лабораторной работы были изучены и реализованы основные алгоритмы и структуры данных, связанные с двоичными деревьями поиска.

В процессе работы были решены задачи на:

- обходы двоичного дерева;
- проверку корректности свойства бинарного дерева поиска;
- вычисление высоты дерева;
- вычисление баланса вершин;
- обработку запросов вставки, удаления, поиска существования, следующего и предыдущего элемента;
- поиск k -го по порядку элемента и k -го максимума.

Для более простых задач использовались стандартные обходы дерева и обычные массивы. Для задач с большим количеством запросов были реализованы более эффективные структуры данных:

- декартово дерево,
- AVL-дерево.

При решении задач на сбалансированные деревья были использованы повороты, пересчёт высоты поддеревьев и хранение дополнительных параметров, таких как размер поддерева. Это позволило обеспечить быструю обработку операций даже при большом количестве запросов.

Все программы были протестированы на нескольких наборах входных данных, включая:

- пустое дерево;
- дерево из одной вершины;
- корректные и некорректные деревья;

- вырожденные деревья;
- последовательности запросов с добавлением, удалением и поиском.

В результате выполнения лабораторной работы были закреплены навыки работы с деревьями поиска, итеративными обходами, проверкой свойств дерева и реализацией сбалансированных структур данных. Все решения соответствуют требованиям задач и укладываются в заданные ограничения по времени и памяти.